Al-Furqan — Implementation Plan v1.0

From Current State to Target Architecture

Project: Al-Furqan (الفرقان) — Axiom-Anchored Reasoning Engine **Document Type:** Implementation Plan — Task-Level Breakdown **Version:** 1.0 **Date:** March 21, 2026 **Based On:** Architecture Document v2.0 (Approved by CTO) **Status:** Ready for Execution **Repository:** https://gitlab.variiance.com/ai/al-furqan

---

Current State Assessment

What Exists (Sprint 2 — Complete ✅)

```
src/al_furqan/
├── api/            ✅ FastAPI (app, routers, schemas, converters, dependencies)
├── auth/           ✅ Auth (key_manager, middleware, security, rate_limiter)
├── core/           ✅ Engine (reasoning_engine, cot, cot_engine, cot_prompts)
├── providers/      ✅ LLM (llm_layer — multi-provider)
├── review/         ✅ Human review (human_review)
├── store/          ✅ Storage (verdict_store — ChromaDB)
├── cli.py          ✅ CLI entry point
└── config.py       ✅ YAML configuration
```

Tests: 205+ passing · **Auth:** bcrypt + API keys · **LLM:** Claude, Qwen, Ollama

What's Missing (Sprint 3-5 Target)

```
src/al_furqan/
├── engine/         ❌ Refactored Engine (gates as separate modules)
│   ├── axioms.py
│   ├── models.py
│   ├── pipeline.py
│   ├── prompts.py
│   ├── gates/
│   ├── chains/
│   └── symbolic/
├── kb/             ❌ Knowledge Base (entire layer)
│   ├── embeddings.py
│   ├── retriever.py
│   ├── knowledge_linker.py
│   ├── collections/
│   ├── graph/
│   └── ingestion/
└── api/
    └── orchestrator.py  ❌ Orchestrator (connects all layers)
```

Sprint 3 — Knowledge Infrastructure

Duration: 4-5 weeks **Goal:** Build the entire Knowledge Base layer. No engine changes. **Dependency:** None (independent layer)

Task 3A — Engine Refactor + Model Metadata

Duration: 3-4 days **Agent:** Agent-1 (Engine Specialist) **Priority:** 🔴 Critical (blocks everything else)

3A.1 — Extract Axioms to Standalone Module

File: `src/al_furqan/engine/axioms.py` **Source:** Extract from `core/reasoning_engine.py` lines ~230-330

```
# What to extract:
FRAMEWORK_PREAMBLE = """..."""   # Move as-is
AXIOMS = """..."""               # Move as-is
GATE_DEFINITIONS = """..."""     # Move as-is
SCORING_RULES = """..."""        # Move as-is

# Add:
AXIOM_VERSION = "2.0"
AXIOM_HASH = sha256(AXIOMS + GATE_DEFINITIONS + SCORING_RULES)
```

Steps:

1. Create `src/al_furqan/engine/` directory structure
2. Copy axiom constants from `reasoning_engine.py` to `axioms.py`
3. Add version string and hash computation
4. Update `reasoning_engine.py` to import from `engine.axioms`
5. Verify all 205 tests still pass

Test: `pytest tests/ -x` — zero failures

3A.2 — Extract Data Models

File: `src/al_furqan/engine/models.py` **Source:** Extract from `core/reasoning_engine.py`

```
# Extract these classes:
class SystemType(Enum): ...
class GateResult(Enum): ...
class GateScore: ...
class Verdict: ...
class DualPerspectiveVerdict: ...
class InformationalResponse: ...
```

Steps:

1. Move all dataclasses and enums to `engine/models.py`
2. Update all imports across the codebase (`api/` , `store/` , `tests/`)
3. Run full test suite

3A.3 — Extract Prompt Templates

File: `src/al_furqan/engine/prompts.py` **Source:** Extract from `core/reasoning_engine.py`

```
# Extract these functions:
def build_scan_prompt(question, context): ...
def build_mirror_prompt(question, scan_result, context): ...
def build_verdict_prompt(question, scan, mirror): ...
def build_correction_prompt(verdict): ...
```

Steps:

1. Move prompt builder functions to `engine/prompts.py`
2. Prompts import axioms from `engine.axioms`
3. Update `reasoning_engine.py` to use new imports
4. Run full test suite

3A.4 — Add Model Metadata Tracking to Verdict

File: `src/al_furqan/engine/models.py`

```
@dataclass
class Verdict:
    # ... existing fields ...

    # NEW: Model tracking (Sprint 3A)
    model_provider: str = ""
    model_name: str = ""
    model_temperature: float = 0.0
    raw_scan_response: str = ""
    raw_mirror_response: str = ""
    raw_verdict_response: str = ""
```

Steps:

1. Add new fields to Verdict dataclass
2. Update `to_dict()` and `from_dict()` to include new fields
3. Update pipeline to populate model info from LLM response
4. Update API schemas to expose model metadata
5. Update verdict_store to persist new fields
6. Run full test suite + add specific tests for model tracking

---

3A.5 — Create Pipeline Module

File: `src/al_furqan/engine/pipeline.py`

```
class EvaluationPipeline:
    """Orchestrates the evaluation flow: Scan → Mirror → Verdict → Correct"""

    def __init__(self, llm_fn: Callable, axioms: str, gates: str):
        self.llm_fn = llm_fn
        self.axioms = axioms
        self.gates = gates

    def evaluate(self, question: str, context: str = "") -> Verdict:
```

```
        scan = self._scan(question, context)
        mirror = self._mirror(question, scan, context)
        verdict = self._verdict(question, scan, mirror)
        return self._correct(verdict)
```

Steps:

1. Extract evaluation logic from `reasoning_engine.py` to `pipeline.py`
2. Pipeline receives LLM function, doesn't own it
3. `reasoning_engine.py` becomes a thin wrapper using Pipeline
4. All existing tests pass without modification

Task 3B — Embedding Infrastructure

Duration: 3-4 days **Agent:** Agent-2 (KB Specialist) **Priority:** 🔴 Critical (blocks 3C) **Dependency:** None

3B.1 — Install and Benchmark CamelBERT

File: `src/al_furqan/kb/embeddings.py`

```python
from sentence_transformers import SentenceTransformer

class EmbeddingModel:
    """Abstraction over embedding models for easy swapping."""

    def __init__(self, model_name: str = "CAMeL-Lab/bert-base-arabic-camelbert-ca"):
        self.model = SentenceTransformer(model_name)
        self.dimension = self.model.get_sentence_embedding_dimension()

    def embed(self, texts: list[str]) -> list[list[float]]:
        return self.model.encode(texts, normalize_embeddings=True).tolist()

    def embed_query(self, query: str) -> list[float]:
        return self.model.encode(query, normalize_embeddings=True).tolist()
```

Steps:

1. Create `src/al_furqan/kb/` directory structure
2. Install `sentence-transformers`, `camel-tools`
3. Implement `EmbeddingModel` with CamelBERT
4. Write benchmark script: embed 1000 Islamic texts, measure time + memory
5. Create test file `tests/test_embeddings.py`

Benchmark targets:

- Embedding 1000 texts < 30s
- Memory usage < 1GB
- Dimension: 768

3B.2 — Add ModernBERT as Alternative

File: `src/al_furqan/kb/embeddings.py`

```
AVAILABLE_MODELS = {
    "camelbert": "CAMeL-Lab/bert-base-arabic-camelbert-ca",
    "modernbert": "nomic-ai/modernbert-embed-base",  # or Arabic variant
    "minilm": "sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2",
}
```

Steps:

1. Add model registry with available models
2. Benchmark ModernBERT on same test set
3. Compare MRR@10 between models
4. Document results in `benchmarks/embedding_comparison.md`
5. Select default model based on results

Task 3C — Knowledge Base Collections

Duration: 8-10 days **Agent:** Agent-3 (Data Engineer) **Priority:** 🔴 Critical **Dependency:** 3B (needs embeddings)

3C.1 — Quran Collection

File: `src/al_furqan/kb/collections/quran.py`

```
class QuranCollection:
    """6,236 verses with Arabic + English + Tafsir."""

    def __init__(self, db_client, embedding_model: EmbeddingModel):
        self.db = db_client
        self.embedder = embedding_model

    def ingest(self, data_path: str) -> int:
        """Ingest Quran data from JSON. Returns verse count."""
        ...

    def search(self, query: str, limit: int = 10) -> list[QuranResult]:
        """Semantic + keyword search across verses."""
        ...

    def get_verse(self, surah: int, ayah: int) -> QuranVerse:
        """Get specific verse by surah:ayah."""
        ...

    def get_context(self, surah: int, ayah: int, window: int = 3) ->
list[QuranVerse]:
        """Get verse with surrounding context."""
        ...
```

Data sources:

- Quran text (AR+EN): `data/quran/quran_complete.json` (to be prepared)
- Jalalayn Tafsir: `data/quran/tafsir_jalalayn.json`

- Muyassar Tafsir: `data/quran/tafsir_muyassar.json`

Steps:

1. Prepare Quran dataset: 6,236 verses with metadata (surah, ayah, juz, hizb, page)
2. Include Arabic text + English translation + 2 tafsir sources
3. Implement collection class with CRUD
4. Implement ingestion pipeline (embed + store)
5. Implement search (semantic + keyword hybrid)
6. Write tests: `tests/test_quran_collection.py`

Test cases:

- Ingest all 6,236 verses
- Search "الربا" → returns Baqara 275-281
- Get verse context: surah=2, ayah=255 → returns Ayat al-Kursi + surrounding

---

3C.2 — Hadith Collection

File: `src/al_furqan/kb/collections/hadith.py`

```
class HadithCollection:
    """38,016+ hadith from 10 collections with grading."""

    def __init__(self, db_client, embedding_model: EmbeddingModel):
        self.db = db_client
        self.embedder = embedding_model

    def ingest(self, data_path: str) -> int:
        """Ingest hadith data. Returns hadith count."""
        ...

    def search(self, query: str, limit: int = 10,
               grading_filter: list[str] = None) -> list[HadithResult]:
        """Search hadith with optional grading filter (sahih, hasan, daif)."""
        ...

    def get_hadith(self, collection: str, number: int) -> Hadith:
        """Get specific hadith by collection + number."""
        ...
```

Data sources:

- 10 collections: Bukhari, Muslim, Tirmidhi, Abu Dawud, Nasa'i, Ibn Majah, Malik, Ahmad, Darimi, Ibn Hibban
- Each hadith: Arabic text, English translation, chain (isnad), grading, topics

Steps:

1. Prepare hadith dataset from existing APIs/datasets
2. Implement grading filter (Sahih > Hasan > Da'if)
3. Implement collection class with search
4. Include narrator chain (isnad) as metadata
5. Write tests: `tests/test_hadith_collection.py`

3C.3 — Fiqh Rules Collection

File: `src/al_furqan/kb/collections/fiqh.py`

```
class FiqhCollection:
    """50+ core fiqh rules with evidence mapping."""

    FIVE_MAJOR_RULES = [
        "الأمور بمقاصدها",           # Matters by their intentions
        "اليقين لا يزول بالشك",      # Certainty not removed by doubt
        "المشقة تجلب التيسير",       # Hardship brings ease
        "الضرر يُزال",               # Harm must be eliminated
        "العادة محكّمة",             # Custom is authoritative
    ]
```

Steps:

1. Compile 50+ fiqh rules with Arabic + English
2. Map each rule to supporting Quran verses and hadith
3. Include application examples
4. Implement collection with search
5. Write tests

3C.4 — Unified Retriever

File: `src/al_furqan/kb/retriever.py`

```
class UnifiedRetriever:
    """Searches across all collections with hybrid strategy."""

    def __init__(self, quran: QuranCollection, hadith: HadithCollection,
                 fiqh: FiqhCollection, reranker=None):
        self.quran = quran
        self.hadith = hadith
        self.fiqh = fiqh
        self.reranker = reranker

    def retrieve(self, query: str, config: RetrievalConfig = None) ->
KnowledgeContext:
        """
        1. Embed query
        2. Search all collections in parallel
        3. Merge + deduplicate results
        4. Rerank (if reranker available)
        5. Format as KnowledgeContext
        """
        ...
```

Steps:

1. Implement parallel search across all collections

2. Merge and deduplicate results by relevance score
3. Implement `KnowledgeContext` data model (formatted text for engine consumption)
4. Add optional cross-encoder reranker
5. Write integration tests: `tests/test_retriever.py`

Task 3D — Knowledge Graph

Duration: 8-10 days **Agent:** Agent-4 (Graph Specialist) **Priority:** 🟡 High **Dependency:** 3C (needs collections populated)

3D.1 — Graph Schema Definition

File: `src/al_furqan/kb/graph/schema.py`

```
# Node types
NODE_TYPES = {
    "Ayah": {"surah": int, "ayah": int, "text_ar": str, "text_en": str, "topics":
list},
    "Hadith": {"collection": str, "number": int, "text_ar": str, "grading": str},
    "FiqhRule": {"text_ar": str, "text_en": str, "category": str},
    "Scholar": {"name": str},
    "Topic": {"name_ar": str, "name_en": str},
    "Maqsad": {"name_ar": str, "name_en": str},  # Maqasid al-Shariah
}

# Edge types
EDGE_TYPES = {
    "INTERPRETED_BY": ("Ayah", "Hadith"),
    "EXPLAINED_BY": ("Ayah|Hadith", "Lesson"),
    "ESTABLISHES": ("Ayah|Hadith", "FiqhRule"),
    "BY_SCHOLAR": ("Lesson", "Scholar"),
    "TAGGED_WITH": ("Ayah|Hadith", "Topic"),
    "SERVES_MAQSAD": ("FiqhRule", "Maqsad"),
    "RELATES_TO": ("Ayah", "Ayah"),
}
```

Steps:

1. Define all node and edge types
2. Create Pydantic models for each
3. Define graph constraints (uniqueness, required fields)
4. Write schema validation tests

3D.2 — Graph Store Implementation

File: `src/al_furqan/kb/graph/store.py`

Option A — SurrealDB:

```
class GraphStore:
    def __init__(self, surreal_client):
        self.db = surreal_client
```

```
    async def create_node(self, node_type: str, data: dict) -> str: ...
    async def create_edge(self, edge_type: str, source: str, target: str, data: dict
= None) -> str: ...
    async def traverse(self, start_node: str, edge_types: list, depth: int = 2) ->
list: ...
    async def find_path(self, start: str, end: str) -> list: ...
```

Option B — Neo4j (contingency):

```
class Neo4jGraphStore:
    def __init__(self, driver):
        self.driver = driver
    # Same interface, different backend
```

Steps:

1. Implement GraphStore with SurrealDB backend
2. Implement CRUD for nodes and edges
3. Implement traversal queries (1-hop, 2-hop, 3-hop)
4. Benchmark: 3-hop query < 500ms
5. If SurrealDB fails benchmark → activate Neo4j contingency
6. Write tests: `tests/test_graph_store.py`

3D.3 — Graph Population Pipeline

File: `src/al_furqan/kb/ingestion/populate_graph.py`

```
async def populate_graph(graph: GraphStore, quran: QuranCollection,
                         hadith: HadithCollection, fiqh: FiqhCollection):
    """
    1. Create Ayah nodes from Quran collection
    2. Create Hadith nodes from Hadith collection
    3. Create FiqhRule nodes from Fiqh collection
    4. Create Topic nodes from tag analysis
    5. Create Maqsad nodes (5 maqasid)
    6. Create edges:
       - Ayah↔Hadith (cross-reference)
       - Ayah/Hadith→FiqhRule (establishes)
       - All→Topic (tagged_with)
       - FiqhRule→Maqsad (serves)
       - Ayah↔Ayah (related verses)
    """
```

Steps:

1. Write ingestion scripts for each node type
2. Implement cross-reference detection (verse ↔ hadith)
3. Implement topic extraction and linking
4. Map fiqh rules to maqasid al-shariah
5. Run full population on complete dataset

6. Verify graph integrity (no orphan nodes, no broken edges)

3D.4 — Knowledge Linker

File: `src/al_furqan/kb/knowledge_linker.py`

```python
class KnowledgLinker:
    """Builds scholarly reasoning chains from graph traversal."""

    def __init__(self, graph: GraphStore, retriever: UnifiedRetriever):
        self.graph = graph
        self.retriever = retriever

    def build_chain(self, query: str, sources: list) -> ReasoningChain:
        """
        Given sources, traverse graph to find:
        1. Related verses/hadith
        2. Connecting fiqh rules
        3. Supporting scholarly interpretations
        4. Build a chain of reasoning
        """
        ...

    def expand_context(self, sources: list, depth: int = 2) -> list:
        """Expand sources by traversing graph relationships."""
        ...
```

Steps:

1. Implement graph-enhanced retrieval
2. Build reasoning chains from traversal results
3. Score chain confidence based on source strength
4. Write tests with real graph data

Task 3E — Sprint 3 Integration Testing

Duration: 3-4 days **Agent:** Agent-5 (QA) **Priority:** 🔴 Critical **Dependency:** 3A-3D complete

Test suite:

```
tests/
├── test_embeddings.py          # Embedding model quality
├── test_quran_collection.py    # Quran CRUD + search
├── test_hadith_collection.py   # Hadith CRUD + search + grading
├── test_fiqh_collection.py     # Fiqh rules + evidence mapping
├── test_retriever.py           # Unified retrieval + reranking
├── test_graph_store.py         # Graph CRUD + traversal
├── test_graph_population.py    # Full graph population
├── test_knowledge_linker.py    # Reasoning chains
└── test_kb_integration.py      # End-to-end KB tests
```

Benchmark targets:

- Quran search < 100ms
- Hadith search < 150ms
- Graph 3-hop traversal < 500ms
- Full retrieval pipeline < 500ms
- KB coverage: 100% of Quran, >95% of Hadith dataset

Sprint 4 — Engine Refinement

Duration: 3-4 weeks **Goal:** Add Z3 symbolic verification + guided chains + deterministic scoring to engine. **Dependency:** Sprint 3A (engine refactor)

Task 4A — Gate Decomposition

Duration: 5-7 days **Agent:** Agent-1 (Engine Specialist) **Priority:** 🔴 Critical

4A.1 — Abstract Gate Base Class

File: `src/al_furqan/engine/gates/base.py`

```
from abc import ABC, abstractmethod

class Gate(ABC):
    """Abstract base class for all survival gates."""

    @property
    @abstractmethod
    def name(self) -> str: ...

    @property
    @abstractmethod
    def description(self) -> str: ...

    @abstractmethod
    def evaluate(self, chain_results: dict) -> GateScore:
        """Deterministic scoring from chain extraction results."""
        ...

    @abstractmethod
    def get_chain_questions(self) -> list[str]:
        """Return guided chain questions for this gate."""
        ...
```

4A.2 — Implement Individual Gates

Files:

- `src/al_furqan/engine/gates/source_integrity.py`
- `src/al_furqan/engine/gates/structural_consistency.py`
- `src/al_furqan/engine/gates/mediation_zeroing.py`
- `src/al_furqan/engine/gates/origin_aware.py`

Each gate:

1. Has specific chain questions (3-5 per gate)
2. Has deterministic scoring function (code, not LLM)
3. Has clear survive/fail threshold
4. Is independently testable

Steps:

1. Implement each gate as separate class extending `Gate`
2. Extract scoring logic from current `reasoning_engine.py`
3. Each gate's `evaluate()` takes structured chain results, returns score
4. Write per-gate unit tests with known inputs → expected scores

Test: Same input → same score, 100% of the time (deterministic)

Task 4B — Guided Reasoning Chains

Duration: 5-7 days **Agent:** Agent-1 (Engine Specialist) **Priority:** 🔴 Critical **Dependency:** 4A

4B.1 — Chain Executor

File: `src/al_furqan/engine/chains/executor.py`

```
class ChainExecutor:
    """Executes guided chain questions through LLM."""

    def __init__(self, llm_fn: Callable):
        self.llm_fn = llm_fn

    def execute_chain(self, question: str, gate: Gate,
                      context: str = "") -> dict:
        """
        For each chain question in the gate:
        1. Build prompt with previous answers as context
        2. Call LLM for structured extraction
        3. Parse response into structured data
        4. Feed to next question
        Returns: dict of extracted facts for scoring
        """
        ...
```

4B.2 — Deterministic Scorer

File: `src/al_furqan/engine/chains/scorer.py`

```
class DeterministicScorer:
    """Pure Python scoring — NO LLM involvement."""

    def score_gate(self, gate: Gate, extractions: dict) -> GateScore:
        """
        Takes extracted facts from chain executor.
        Applies deterministic scoring rules.
        Returns GateScore with exact numeric score.
        """
```

```
        # Example for Source Integrity:
        base_score = SOURCE_TYPE_SCORES[extractions["source_type"]]
        # divine=90, prophetic=80, scholarly=60, human_theory=40
        if extractions["is_verifiable"]:
            base_score *= 1.0
        else:
            base_score *= 0.5
        if extractions["contradicts_primary"]:
            base_score -= 40
        ...
```

Steps:

1. Define scoring rules as code (not prompts)
2. Implement scorer for each gate
3. Write exhaustive test cases: every combination of inputs → known output
4. Prove determinism: run 100 times, verify identical scores

Task 4C — Z3 Symbolic Verification

Duration: 7-10 days **Agent:** Agent-6 (Symbolic AI Specialist) **Priority:** 🟡 High **Dependency:** 4A, 4B

4C.1 — Formal Axiom Encoding

File: `src/al_furqan/engine/symbolic/formal_axioms.py`

```
from z3 import *

# Core sorts
Entity = DeclareSort('Entity')
Framework = DeclareSort('Framework')

# Core predicates
Exists = Function('Exists', Entity, BoolSort())
HasPurpose = Function('HasPurpose', Entity, BoolSort())
HasDesign = Function('HasDesign', Entity, BoolSort())
HasTranscendentSource = Function('HasTranscendentSource', Framework, BoolSort())
IsContingent = Function('IsContingent', Framework, BoolSort())
PreservesNatural = Function('PreservesNatural', Framework, BoolSort())

# Axiom 1: Design
x = Const('x', Entity)
axiom_design = ForAll([x], Implies(Exists(x), HasPurpose(x)))

# Axiom 2: Network Effect
HasCausalNetwork = Function('HasCausalNetwork', Entity, BoolSort())
axiom_network = ForAll([x], Implies(Exists(x), HasCausalNetwork(x)))

# Axiom 3: Alignment
Aligned = Function('Aligned', Entity, BoolSort())
Functional = Function('Functional', Entity, BoolSort())
```

```
axiom_alignment = ForAll([x], Implies(Exists(x),
    Aligned(x) == Functional(x)))
```

4C.2 — Predicate Extractor

File: `src/al_furqan/engine/symbolic/predicate_extractor.py`

```
class PredicateExtractor:
    """Extracts Z3-compatible predicates from chain results."""

    def extract(self, chain_results: dict) -> list[BoolRef]:
        """
        Takes structured chain results.
        Maps them to Z3 predicates.
        Returns list of Z3 boolean expressions.
        """
        predicates = []
        if chain_results.get("source_type") == "divine":
            predicates.append(HasTranscendentSource(framework))
        if chain_results.get("preserves_five_necessities"):
            predicates.append(PreservesNatural(framework))
        ...
        return predicates
```

4C.3 — Z3 Verifier

File: `src/al_furqan/engine/symbolic/verifier.py`

```
class SymbolicVerifier:
    """Formal verification using Z3 SMT solver."""

    def __init__(self):
        self.axioms = load_formal_axioms()

    def verify(self, predicates: list[BoolRef]) -> VerificationResult:
        """
        1. Create solver with axioms
        2. Add extracted predicates
        3. Check satisfiability
        4. Return result with proof/disproof
        """
        solver = Solver()
        for axiom in self.axioms:
            solver.add(axiom)
        for pred in predicates:
            solver.add(pred)

        result = solver.check()
        if result == sat:
            return VerificationResult(
                consistent=True,
```

```
                proof=str(solver.model()),
            )
        elif result == unsat:
            core = solver.unsat_core()
            return VerificationResult(
                consistent=False,
                contradictions=core,
                proof="UNSAT — formal contradiction detected",
            )
        else:
            return VerificationResult(
                consistent=None,  # unknown
                proof="Z3 timeout — falling back to rule engine",
            )
```

Steps:

1. Encode all axioms in Z3
2. Implement predicate extraction from chain results
3. Implement verifier with sat/unsat/unknown handling
4. Add timeout handling (default: 10s per verification)
5. Implement hybrid fallback for `unknown` results
6. Write tests with known consistent/inconsistent inputs
7. Benchmark: verification < 5s for typical queries

Task 4D — Human Feedback Loop

Duration: 3-4 days **Agent:** Agent-1 (Engine Specialist) **Priority:** 🟢 Medium **Dependency:** 4A-4C

4D.1 — Feedback Data Model

File: `src/al_furqan/store/feedback_store.py`

```
@dataclass
class HumanFeedback:
    verdict_id: str
    reviewer: str
    rating: str  # "correct", "partially_correct", "incorrect"
    gate_corrections: dict  # {"Source Integrity": {"score": 50, "reason": "..."}}
    notes: str
    timestamp: float
```

Steps:

1. Implement feedback storage (JSON + ChromaDB)
2. Link feedback to verdicts
3. API endpoints for submitting feedback
4. Dashboard for reviewing feedback patterns
5. Tests for feedback CRUD

Task 4E — Sprint 4 Testing

Duration: 3-4 days **Agent:** Agent-5 (QA)

```
tests/
├── test_gate_source_integrity.py
├── test_gate_structural_consistency.py
├── test_gate_mediation_zeroing.py
├── test_gate_origin_aware.py
├── test_chain_executor.py
├── test_deterministic_scorer.py
├── test_predicate_extractor.py
├── test_symbolic_verifier.py
├── test_z3_axioms.py
└── test_feedback.py
```

Determinism test: Run 50 evaluations × 3 times each. Score must be identical all 3 runs.

Sprint 5 — Integration

Duration: 3-4 weeks **Goal:** Connect Engine ↔ KB ↔ Storage through Orchestrator. **Dependency:** Sprint 3 + Sprint 4

Task 5A — Orchestrator

Duration: 5-7 days **Agent:** Agent-1 (Engine Specialist) **Priority:** 🔴 Critical

5A.1 — Core Orchestrator

File: `src/al_furqan/api/orchestrator.py`

```python
class Orchestrator:
    """The ONLY component that knows about all layers."""

    def __init__(self, engine: EvaluationPipeline,
                 kb: UnifiedRetriever,
                 graph: GraphStore,
                 linker: KnowledgeLinker,
                 store: VerdictStore):
        self.engine = engine
        self.kb = kb
        self.graph = graph
        self.linker = linker
        self.store = store

    async def evaluate(self, question: str,
                       use_kb: bool = False) -> Verdict:
        """Full evaluation pipeline."""
        context = ""
        if use_kb:
            # 1. Retrieve from KB
            kb_result = self.kb.retrieve(question)
            # 2. Expand via graph
            expanded = self.linker.expand_context(kb_result.sources)
```

```
            # 3. Build reasoning chain from graph
            chain = self.linker.build_chain(question, expanded)
            # 4. Format context for engine
            context = chain.formatted_text

        # 5. Engine evaluates with context
        verdict = self.engine.evaluate(question, context=context)

        # 6. Add source citations
        if use_kb:
            verdict.source_citations = kb_result.citations

        # 7. Store verdict
        self.store.store(verdict)

        return verdict

    async def evaluate_grounded(self, question: str) -> Verdict:
        """Always use KB — the target flow."""
        return await self.evaluate(question, use_kb=True)
```

5A.2 — New API Endpoint: Grounded Evaluation

File: `src/al_furqan/api/routers/evaluate.py` (update)

```
@router.post("/evaluate-grounded")
async def evaluate_grounded(request: EvaluateRequest,
                            orchestrator: Orchestrator = Depends()):
    """Evaluate with full KB grounding + graph expansion."""
    verdict = await orchestrator.evaluate_grounded(request.question)
    return convert_verdict(verdict)
```

Task 5B — Comparative Testing Framework

Duration: 5-7 days **Agent:** Agent-5 (QA) **Priority:** 🔴 Critical **Dependency:** 5A

5B.1 — Side-by-Side Evaluation

File: `tests/test_comparative.py`

```
BENCHMARK_QUESTIONS = [
    {
        "question": "Is fractional reserve banking ethical?",
        "expected_gates": {"G1": "Fail", "G2": "Fail", "G3": "Fail", "G4": "Fail"},
        "expected_score_range": (0, 30),
    },
    {
        "question": "Is zakat an effective wealth distribution mechanism?",
        "expected_gates": {"G1": "Survive", "G2": "Survive", "G3": "Survive", "G4":
"Survive"},
        "expected_score_range": (80, 100),
```

```
    },
    # ... 16 more benchmark questions covering all system types
]

async def test_grounded_vs_ungrounded():
    """Compare verdicts with and without KB grounding."""
    for benchmark in BENCHMARK_QUESTIONS:
        v_plain = await orchestrator.evaluate(benchmark["question"])
        v_grounded = await orchestrator.evaluate_grounded(benchmark["question"])

        # Grounded should:
        # 1. Have source citations (plain doesn't)
        assert len(v_grounded.source_citations) > 0
        # 2. Have consistent gate results
        assert_gates_match(v_grounded, benchmark["expected_gates"])
        # 3. Score in expected range
        assert benchmark["expected_score_range"][0] <= v_grounded.total_score
```

5B.2 — Cross-Model Consistency

File: `tests/test_cross_model.py`

```
MODELS = ["claude-sonnet-4-20250514", "qwen/qwen-2.5-72b", "ollama/qwen2.5:14b"]

async def test_cross_model_consistency():
    """Same question, different models → scores within 15-point range."""
    for question in BENCHMARK_QUESTIONS[:5]:
        scores = []
        for model in MODELS:
            verdict = await evaluate_with_model(question, model)
            scores.append(verdict.total_score)

        score_range = max(scores) - min(scores)
        assert score_range <= 15, f"Score variance too high: {scores}"
```

Task 5C — Pattern Learning (Store)

Duration: 4-5 days **Agent:** Agent-2 (KB Specialist) **Priority:** 🟢 Medium **Dependency:** 5A

5C.1 — Pattern Store

File: `src/al_furqan/store/pattern_store.py`

```
class PatternStore:
    """Stores successful reasoning patterns for reuse."""

    def extract_pattern(self, verdict: Verdict) -> Pattern:
        """Extract generalizable pattern from a verified verdict."""
        ...

    def find_similar(self, question: str, threshold: float = 0.8) -> list[Pattern]:
```

```
        """Find patterns similar to a new question."""
        ...

    def apply_pattern(self, pattern: Pattern, question: str) -> PreliminaryVerdict:
        """Apply a known pattern to a new question."""
        ...
```

Task 5D — API Finalization

Duration: 3-4 days **Agent:** Agent-1 (Engine Specialist)

1. Update all API schemas for new fields (citations, Z3 proof, chain results)
2. Add `/api/v1/sources/search` endpoint (direct KB search)
3. Update `/api/v1/stats` with KB statistics
4. Update OpenAPI documentation
5. Performance optimization (caching, connection pooling)

Task 5E — Sprint 5 Integration Testing

Duration: 5-7 days **Agent:** Agent-5 (QA)

```
tests/
├── test_orchestrator.py              # Full pipeline tests
├── test_comparative.py               # Grounded vs ungrounded
├── test_cross_model.py               # Multi-model consistency
├── test_end_to_end.py                # User question → full verdict
├── test_edge_cases.py                # 18 edge cases from architecture
├── test_performance.py               # Latency benchmarks
└── test_pattern_learning.py          # Pattern extraction + reuse
```

Final benchmark targets:

- Full grounded evaluation < 15s
- Quick evaluation (no KB) < 5s
- KB retrieval < 500ms
- Z3 verification < 5s
- Cross-model score variance < 15 points
- 100% deterministic scoring (same extracted facts → same score)

Agent Assignment Summary

Agent	Role	Sprints	Key Files
Agent-1	Engine Specialist	3A, 4A-4D, 5A, 5D	engine/, pipeline, orchestrator
Agent-2	KB Specialist	3B, 5C	kb/embeddings, pattern_store
Agent-3	Data Engineer	3C	kb/collections/, kb/ingestion/
Agent-4	Graph Specialist	3D	kb/graph/, knowledge_linker
Agent-5	QA Engineer	3E, 4E, 5B, 5E	tests/

Agent-6	Symbolic AI	4C	engine/symbolic/

Execution Order (Critical Path)

```
Week 1-2:
  Agent-1: 3A (Engine Refactor) ─── BLOCKS EVERYTHING
  Agent-2: 3B (Embeddings) ──────── parallel, no dependency
  Agent-3: Prepare datasets ─────── parallel, no dependency

Week 3-4:
  Agent-3: 3C (Collections) ─────── needs 3B
  Agent-4: 3D.1-3D.2 (Graph Schema + Store) ── parallel

Week 5:
  Agent-4: 3D.3-3D.4 (Population + Linker) ── needs 3C
  Agent-1: 4A (Gate Decomposition) ── needs 3A
  Agent-5: 3E (Sprint 3 Tests) ──── needs 3C, 3D

Week 6-7:
  Agent-1: 4B (Chains + Scorer)
  Agent-6: 4C (Z3) ──────────────── parallel with 4B

Week 8:
  Agent-1: 4D (Feedback)
  Agent-5: 4E (Sprint 4 Tests)

Week 9-10:
  Agent-1: 5A (Orchestrator) ─────── needs Sprint 3 + 4
  Agent-5: 5B (Comparative Tests)

Week 11-12:
  Agent-2: 5C (Pattern Learning)
  Agent-1: 5D (API Finalization)
  Agent-5: 5E (Final Integration Tests)
```

Multi-Agent Execution Strategy

Parallel Streams

```
Stream A (Engine):     3A ──── 4A ── 4B ──── 5A ── 5D
Stream B (KB):         3B ── 3C ──────────────── 5C
Stream C (Graph):           3D ──────────────────
Stream D (Symbolic):              4C ────────────
Stream E (QA):              3E ──── 4E ──── 5B ── 5E
Stream F (Data):       Datasets ── 3C (support) ──
```

Rules

1. **No agent touches another agent's files** without coordination

2. **Interface contracts** defined before implementation starts
3. **Daily sync:** agents report blockers and progress
4. **Integration tests** run after every agent completes a task
5. **All code must pass existing 205 tests** before merge

This implementation plan is the execution blueprint for Architecture v2.0. Every task maps directly to a section in the architecture document. Ready for multi-agent execution.